

Documentación Técnica del Proyecto Bank Account Application

Decisiones de Diseño y Guía de Ejecución

Prueba Técnica — Desarrollador Full Stack

Fiduciaria Aval

Julio de 2026

Tabla de contenido

Resumen	4
Introducción	4
Decisiones Técnicas y Arquitectónicas	5
Stack Tecnológico del Backend	5
Arquitectura por Capas	5
Principios SOLID	6
Base de Datos y Perfiles de Configuración	6
Validaciones, Excepciones y Logging	6
Frontend y Microfrontends	7
Contenedorización y Despliegue	7
<i>Arquitectura Cloud con API Gateway</i>	7
Control de Versiones	8
Diagrama de Despliegue	9
Guía de Ejecución del Proyecto	10
Requisitos Previos	10
Ejecución en Desarrollo Local	10
Ejecución con Docker	10
Pruebas Unitarias	11
Endpoints Principales de la API	11

Conclusión.....	12
Referencias	13

Resumen

El presente documento describe de forma breve las decisiones técnicas y arquitectónicas adoptadas en el desarrollo del sistema Bank Account Application, compuesto por un microservicio backend en Spring Boot 3 y un portal frontend en React con arquitectura de microfrontends. Asimismo, se detallan los procedimientos para ejecutar el proyecto en entorno de desarrollo local y mediante contenedores Docker. El objetivo es facilitar la comprensión, evaluación y despliegue de la solución por parte de revisores técnicos y evaluadores de la prueba.

Introducción

Bank Account Application es una solución full stack orientada a la gestión de cuentas bancarias. Permite crear cuentas, realizar consignaciones y retiros, y consultar saldos mediante una API REST documentada con Swagger. El frontend expone estas operaciones a través de un portal web modular. La solución fue diseñada aplicando principios SOLID, patrones de diseño reconocidos y buenas prácticas de desarrollo empresarial.

Decisiones Técnicas y Arquitectónicas

Stack Tecnológico del Backend

Se seleccionó Java 17 como versión del lenguaje por ser la versión LTS recomendada para aplicaciones empresariales modernas. Spring Boot 3.2.5 fue elegido por su madurez, integración con el ecosistema Spring y soporte nativo para Java 17. Gradle se utilizó como herramienta de construcción por su flexibilidad y por incluir wrapper en el repositorio, lo que garantiza builds reproducibles sin dependencias externas.

- Java 17 y Spring Boot 3.2.5 para el microservicio REST.
- Gradle 8.x con wrapper incluido para compilación y empaquetado.
- Spring Data JPA e Hibernate para persistencia de datos.
- SpringDoc OpenAPI 2.5.0 para documentación interactiva (Swagger).
- JUnit 5 y Mockito para pruebas unitarias.
- Lombok para reducir código repetitivo en entidades y DTOs.

Arquitectura por Capas

El backend sigue una arquitectura por capas con separación estricta de responsabilidades. La capa controller expone los endpoints HTTP; la capa service encapsula la lógica de negocio; la capa repository abstrae el acceso a datos; y los mappers convierten entre entidades JPA y DTOs de la API. Esta organización facilita el mantenimiento, las pruebas y la extensión del sistema.

- Patrón Repository para abstracción de persistencia.
- Patrón DTO para separar el contrato de la API del modelo de dominio.
- Patrón Builder (Lombok) para construcción de objetos.
- Inyección de dependencias por constructor (`@RequiredArgsConstructor`).

Principios SOLID

- Responsabilidad única: cada clase tiene un propósito definido (Controller, Service, Repository, Mapper).
- Abierto/cerrado: extensible mediante interfaces como AccountService.
- Sustitución de Liskov: AccountServiceImpl es intercambiable con su interfaz.
- Segregación de interfaces: contratos definidos por operación de negocio.
- Inversión de dependencias: las capas superiores dependen de abstracciones.

Base de Datos y Perfiles de Configuración

Se definieron dos perfiles de Spring Boot para adaptar la configuración al entorno de ejecución. El perfil default utiliza H2 en memoria, ideal para desarrollo local por su simplicidad y cero configuración. El perfil prod utiliza PostgreSQL, adecuado para entornos de despliegue donde se requiere persistencia de datos. Esta decisión evita instalar bases de datos externas durante el desarrollo, sin sacrificar robustez en producción.

- application.yml: perfil default con H2, consola H2 habilitada y logs DEBUG.
- application-prod.yml: perfil prod con PostgreSQL y credenciales por variables de entorno.

Validaciones, Excepciones y Logging

Las validaciones de entrada se implementaron con Bean Validation (@Valid, @NotBlank, @Positive). Las excepciones de negocio (cuenta no encontrada, saldo insuficiente) se manejan centralizadamente mediante GlobalExceptionHandler, retornando respuestas HTTP estructuradas. Se incorporaron logs con prefijos identificables ([CREAR_CUENTA], [DEPOSITO], [RETIRO], [ERROR-400]) tanto en backend como en frontend para facilitar el diagnóstico de fallos.

Frontend y Microfrontends

El frontend se desarrolló con React 18, TypeScript y Vite. Se adoptó una arquitectura de microfrontends en monorepo para dividir la interfaz por funcionalidad de negocio, alineada con el enfoque modular del backend. Un Shell App orquesta los módulos; el paquete shared centraliza el cliente API, tipos y estilos comunes.

- mf-account-creation: creación de cuentas bancarias.
- mf-deposit: consignaciones.
- mf-withdraw: retiros.
- mf-balance-inquiry: consulta de saldo.
- @bank/shared: API client, tipos TypeScript y estilos compartidos.

Contenedorización y Despliegue

Para el punto extra de despliegue, se implementó contenedorización con Docker multi-stage, reduciendo el tamaño de las imágenes finales. Docker Compose orquesta tres servicios: PostgreSQL, backend Spring Boot y frontend Nginx. Nginx actúa como reverse proxy, redirigiendo las peticiones /accounts hacia el microservicio. Se documentó un pipeline CI/CD propuesto y una arquitectura cloud en AWS.

Arquitectura Cloud con API Gateway

En producción cloud se propone colocar un API Gateway como capa de entrada de la API REST, delante del Application Load Balancer (ALB). El gateway no reemplaza al balanceador: complementa su función. Mientras el ALB distribuye tráfico entre instancias del backend en ECS, el API Gateway centraliza el gobierno de la API pública.

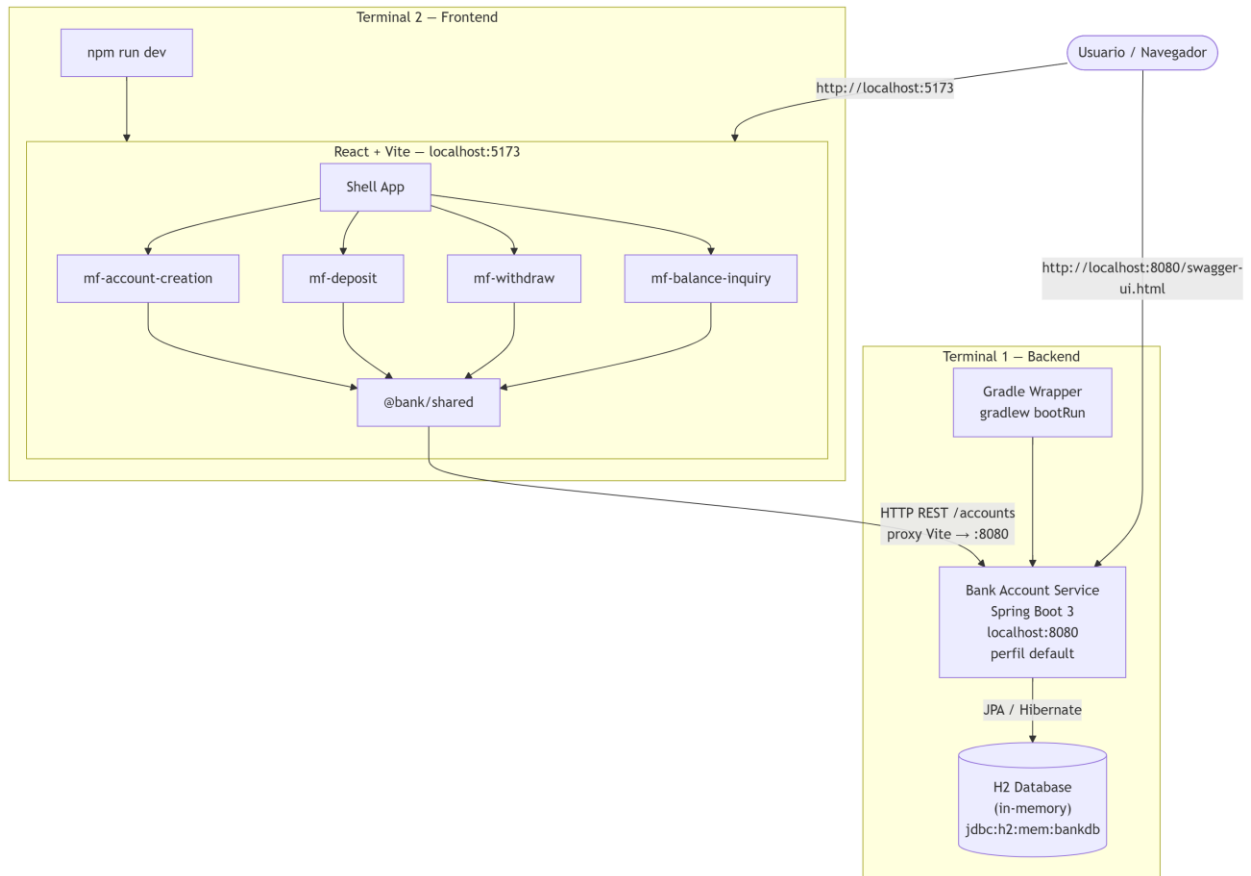
Flujo de una petición API: Cliente → Route 53 → API Gateway → ALB → ECS (Spring Boot) → RDS. Flujo del portal web: Cliente → CloudFront/S3 (React estático); las llamadas a /accounts se enrutan hacia el API Gateway.

- API Gateway: autenticación, rate limiting, CORS, versionado y punto único de entrada (/accounts).
- ALB: balanceo de carga y health checks entre contenedores del backend.
- ECS: ejecución del microservicio Spring Boot.
- RDS: persistencia PostgreSQL en Multi-AZ.
- CloudFront + S3: distribución del frontend React compilado.

Control de Versiones

El repositorio utiliza una estrategia de ramas con main (producción), develop (integración), stage (preproducción) y Feature-JFSL (rama de desarrollo de la prueba técnica). Esta estructura facilita flujos de integración continua y revisión por pull request.

Diagrama de Despliegue



Guía de Ejecución del Proyecto

Requisitos Previos

- Desarrollo local: JDK 17+, Node.js 18+, npm 9+.
- Docker: Docker Desktop 4.x+ y Docker Compose v2.
- Opcional: Postman para pruebas de API (colección incluida en postman/).

Ejecución en Desarrollo Local

Backend (Terminal 1):

```
cd BankAccountApplication
```

```
.\gradlew.bat build
```

```
.\gradlew.bat bootRun
```

El backend estará disponible en <http://localhost:8080>. Swagger UI:
<http://localhost:8080/swagger-ui.html>

Frontend (Terminal 2):

```
cd BankAccountApplication\frontend
```

```
npm install
```

```
npm run dev
```

El portal web estará disponible en <http://localhost:5173>. Vite redirige /accounts al backend en el puerto 8080.

Ejecución con Docker

Desde la raíz del proyecto:

```
cd BankAccountApplication
```

```
docker compose up --build -d
```

Servicios disponibles tras el despliegue:

- Portal web: `http://localhost`
- API REST: `http://localhost:8080/accounts`
- Swagger: `http://localhost:8080/swagger-ui.html`
- PostgreSQL: `localhost:5432` (usuario: bank, contraseña: bank)

Para detener: `docker compose down`. Para eliminar datos: `docker compose down -v`

Pruebas Unitarias

```
cd BankAccountApplication
```

```
.\gradlew.bat test
```

Las pruebas cubren creación de cuenta, retiro exitoso, retiro con saldo insuficiente y retiro con cuenta inexistente.

Endpoints Principales de la API

- POST `/accounts` — Crear cuenta bancaria.
- POST `/accounts/{id}/deposit` — Realizar consignación.
- POST `/accounts/{id}/withdraw` — Realizar retiro.
- GET `/accounts/{id}/balance` — Consultar saldo.

Conclusión

Bank Account Application integra un microservicio backend robusto con un frontend modular, documentación de API, pruebas unitarias, contenedorización y guías de despliegue. Las decisiones adoptadas priorizan la mantenibilidad, la separación de responsabilidades y la facilidad de ejecución tanto en desarrollo como en entornos containerizados, cumpliendo con los requisitos de la prueba técnica full stack.

Referencias

American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). American Psychological Association.

Spring. (2024). Spring Boot reference documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>

React. (2024). React documentation. <https://react.dev/>

Docker Inc. (2024). Docker documentation. <https://docs.docker.com/>

Amazon Web Services. (2024). Amazon API Gateway developer guide. <https://docs.aws.amazon.com/apigateway/>

PostgreSQL Global Development Group. (2024). PostgreSQL documentation. <https://www.postgresql.org/docs/>